

theorem

Vera: Weakest Preconditions for TICL over Rust

From Imp Structural Lemmas to Rust VCGen Rules

Lef Ioannidis

2026

Background: TICL over Imp

TICL Structural Lemmas for Imp

Core idea (from the TICL paper)

For a small imperative language `StImp` with mutable state, the entailment $[t, m \Vdash_L \varphi]_S$ is proved structurally via inference rules on the syntax of t .

Key rules from `imp-lemmas.tex`:

- **Seq**_SAU_L: sequential composition decomposes into bind
- **If**_SAU_L: conditional branches both satisfy the obligation
- **While**_SAG: infinite loop with coinductive invariant $\mathcal{R}$
- **While**_SAU_L: terminating loop with ranking function f

Goal: Lift these to Rust syntax $\rightarrow$ define `wp` for Vera.

Notation

$\sigma \in \text{State}$ (mutable references, heap)

$[t, \sigma \Vdash_L \varphi] \triangleq$ “Rust term t from state σ satisfies φ ”

$\text{wp}(t, \varphi) \triangleq$ weakest precondition on σ s.t. $[t, \sigma \Vdash_L \varphi]$

Temporal formulas

- $\varphi ::= \text{now } P \mid \text{AG } \varphi \mid \varphi \text{ AU } \varphi' \mid \varphi \text{ AN } \varphi' \mid \varphi \wedge \varphi' \mid \varphi \vee \varphi' \mid \varphi \implies \varphi'$
- Sugar: $\text{AF } \varphi' = \top \text{ AU } \varphi'$, $\text{AX } \varphi' = \top \text{ AN } \varphi'$
- $\text{now } P$ where $P : \sigma \rightarrow \mathbb{P}$ — state predicate on current state
- $\text{done } Q$ where $Q : \tau \rightarrow \sigma \rightarrow \mathbb{P}$ — predicate on return value and final state

Comparison with Hoare Logic

Total correctness Hoare triple

$$\{P\} c \{Q\} \iff \forall \sigma. P \sigma \rightarrow [c, \sigma \Vdash_L \text{AF done } Q]$$

Hoare logic is a special case of TIDL:

- $\text{AF done } Q = \top \text{ AU done } Q$: trivial path, just reach Q
- P is the precondition (state predicate on σ)
- Q is the postcondition (predicate on return value and final state)

What TIDL adds beyond Hoare logic

$\text{AG } \varphi$ Safety invariant (infinite computation)

$\varphi \text{ AU done } Q$ Path property φ until Q

$\text{AG AF now } P$ Fairness / recurrence

$\varphi \text{ AN done } Q$ One-step (next)

Inference Rules (Single-Process)

Assignment

Rust: $*x = e$

$$\frac{\begin{array}{l} [e, \sigma \Vdash_R \varphi \text{ AU done } \mathcal{R}] \\ \forall v, \sigma', \mathcal{R} \ v \ \sigma' \rightarrow [\text{skip}, \sigma'[x \mapsto v] \Vdash_L \varphi \text{ AU } \varphi'] \end{array}}{[*x = e, \sigma \Vdash_L \varphi \text{ AU } \varphi']} \text{ASSIGNAU}_L$$

Evaluating e may take steps; φ must hold along the path.

Sequential Composition (let-binding)

Rust: `let x = t; u`

$$\frac{[t, \sigma \Vdash_R \varphi \text{ AU done } \mathcal{R}] \quad \forall x, \sigma', \mathcal{R} \ x \ \sigma' \rightarrow [u[x], \sigma' \Vdash_L \varphi \text{ AU } \varphi']}{[\text{let } x = t; u, \sigma \Vdash_L \varphi \text{ AU } \varphi']} \text{LETAU}_L$$

Direct lift of **BindAU**_L / **SeqS**AU_L from TACL.

Rust: `if c {t} else {u}`

$$\frac{\begin{array}{l} [c, \sigma \Vdash_R \varphi \text{ AU done} = (b, \sigma')] \\ \left\{ \begin{array}{ll} [t, \sigma' \Vdash_L \varphi \text{ AU } \varphi'], & \text{if } b \\ [u, \sigma' \Vdash_L \varphi \text{ AU } \varphi'], & \text{otherwise} \end{array} \right. \end{array}}{[\text{if } c \{t\} \text{ else } \{u\}, \sigma \Vdash_L \varphi \text{ AU } \varphi']} \text{IFAU}_L$$

Evaluating c may take multiple steps; φ must hold along the path.

While Loop (AU – terminating)

Rust: `while c {t}` **with** invariant $\mathcal{R}$, decreases f

$$\frac{\begin{array}{c} \mathcal{R} \sigma \\ \forall \sigma, \mathcal{R} \sigma \rightarrow [c, \sigma \Vdash_R \varphi \text{ AU done} = (b, \sigma')] \\ \wedge \left\{ \begin{array}{l} [t, \sigma' \Vdash_L \varphi \text{ AU } \varphi'] \vee \\ [t, \sigma' \Vdash_R \varphi \text{ AU done } (\lambda \sigma''. \mathcal{R} \sigma'' \wedge f \sigma'' < f \sigma)], \text{ if } b \\ \varphi'(\sigma'), \end{array} \right. \end{array} \quad \text{otherwise}}{[\text{while } c \{t\}, \sigma \Vdash_L \varphi \text{ AU } \varphi']} \text{WHILEAU}_L$$

While Loop (AG – infinite)

Rust: `while c {t}` **with** invariant $\mathcal{R}$ (**no** decreases)

$$\frac{\begin{array}{l} \mathcal{R} \sigma \\ \forall \sigma, \mathcal{R} \sigma \rightarrow \\ [\text{while } c \{t\}, \sigma \Vdash_L \varphi] \wedge \\ [c, \sigma \Vdash_R \varphi \text{ AU done} = (\text{true}, \sigma)] \wedge \\ [t, \sigma \Vdash_R \varphi \text{ AU done } (\lambda \sigma' \Rightarrow \mathcal{R} \sigma')] \end{array}}{[\text{while } c \{t\}, \sigma \Vdash_L \text{AG } \varphi]} \text{WHILEAG}$$

Coinductive hypothesis $[\text{while } c \{t\}, \sigma \Vdash_L \varphi]$; condition must always be true.

Function Call

Rust: $f(e_1, \dots, e_n)$

$$\frac{\begin{array}{c} [e_1, \sigma_0 \Vdash_R \varphi \text{ AU done } \mathcal{R}_1] \\ \forall v_1, \sigma_1. \mathcal{R}_1 v_1 \sigma_1 \rightarrow [e_2, \sigma_1 \Vdash_R \varphi \text{ AU done } \mathcal{R}_2] \\ \dots \\ \forall v_n, \sigma_n. \mathcal{R}_n v_n \sigma_n \rightarrow [f(v_1, \dots, v_n), \sigma_n \Vdash_R \varphi \text{ AU done } \mathcal{R}] \end{array}}{[f(e_1, \dots, e_n), \sigma_0 \Vdash_R \varphi \text{ AU done } \mathcal{R}]} \text{CALLAU}_L$$

Arguments evaluated left-to-right; $\mathcal{R}_i$ chains σ_i into e_{i+1} . Call produces result via $\text{done } \mathcal{R}$; caller binds it with let .

Single-Process Weakest Preconditions

wp: Assignment

$$\begin{aligned} \text{wp}(*x = e, \varphi \text{ AU } \varphi') &= \lambda \sigma. \text{wp}(e, \varphi \text{ AU done } \mathcal{R}) \sigma \\ &\quad \wedge \forall v, \sigma'. \mathcal{R} \ v \ \sigma' \rightarrow \text{wp}(\text{skip}, \varphi \text{ AU } \varphi') (\sigma'[x \mapsto v]) \end{aligned}$$

wp: Let-binding (Sequential Composition)

$$\begin{aligned} \text{wp}(\text{let } x = t; u, \varphi \text{ AU } \varphi') &= \lambda \sigma. \text{wp}(t, \varphi \text{ AU done } \mathcal{R}) \sigma \\ &\quad \wedge \forall x, \sigma'. \mathcal{R} \ x \ \sigma' \rightarrow \text{wp}(u[x], \varphi \text{ AU } \varphi') \sigma' \end{aligned}$$

$$\text{wp}(\text{if } c \{t\} \text{ else } \{u\}, \varphi \text{ AU } \varphi') = \lambda \sigma.$$

$$\text{wp}(c, \varphi \text{ AU done } \mathcal{R}) \sigma \wedge \forall b, \sigma'. \mathcal{R} b \sigma' \rightarrow$$

$$\begin{cases} \text{wp}(t, \varphi \text{ AU } \varphi') \sigma', & \text{if } b \\ \text{wp}(u, \varphi \text{ AU } \varphi') \sigma', & \text{otherwise} \end{cases}$$

wp: While Loop (AU)

$$\text{wp}(\text{while } c \{t\}, \varphi \text{ AU } \varphi') = \lambda \sigma.$$

$$\mathcal{R} \sigma \wedge \forall \sigma, \mathcal{R} \sigma \rightarrow$$

$$\text{wp}(c, \varphi \text{ AU done } \mathcal{R}') \sigma \wedge \forall b, \sigma'. \mathcal{R}' b \sigma' \rightarrow$$

$$\begin{cases} \text{wp}(t, \varphi \text{ AU } \varphi') \sigma' \vee \\ \text{wp}(t, \varphi \text{ AU done } (\lambda _ \sigma''. \mathcal{R} \sigma'' \wedge f \sigma'' < f \sigma)) \sigma', & \text{if } b \\ \varphi'(\sigma'), & \text{o.w.} \end{cases}$$

wp: While Loop (AG)

$$\text{wp}(\text{while } c \{t\}, \text{AG } \varphi) = \lambda \sigma.$$

$$\mathcal{R} \sigma \wedge \forall \sigma, \mathcal{R} \sigma \rightarrow$$

$$\text{wp}(\text{while } c \{t\}, \varphi) \sigma \wedge$$

$$\text{wp}(c, \varphi \text{ AU done} = (\text{true}, \sigma)) \sigma \wedge$$

$$\text{wp}(t, \varphi \text{ AU done } (\lambda \sigma'. \mathcal{R} \sigma')) \sigma$$

Coinductive: $\text{wp}(\text{while } c \{t\}, \varphi) \sigma$ is the coinductive hypothesis.

wp: Function Call – f ensures AF done $\mathcal{R}_f$

f has terminating postcondition

$$\text{wp}(f(e_1, \dots, e_n), \varphi \text{ AU done } \mathcal{R}) = \lambda \sigma_0.$$

$$\text{wp}(e_1, \varphi \text{ AU done } \mathcal{R}_1) \sigma_0 \wedge \forall v_1, \sigma_1. \mathcal{R}_1 v_1 \sigma_1 \rightarrow$$

...

$$\forall v_n, \sigma_n. \mathcal{R}_n v_n \sigma_n \rightarrow \text{wp}(f(v_1, \dots, v_n), \varphi \text{ AU done } \mathcal{R}) \sigma_n$$

f has temporal ensures ψ : transparent checking

Additionally assert $\psi \implies \varphi$ where φ is the caller's obligation.

f ensures AG ψ : cannot prove AF for it

If f has AG ψ (diverges), then $\text{wp}(f(\vec{e}), \text{AF } \varphi')$ is **unprovable** – a diverging call can never reach the AF goal.

Vera Implementation

Vera extends Verus with temporal postconditions and loop annotations:

Annotation	TICL	Proof obligation
<code>ensures af(done(Q))</code>	$\text{AF done } Q$	Loop + decreases
<code>ensures ag(P)</code>	$\text{AG } \varphi$	Infinite loop + invariant
<code>ensures au(P, done(Q))</code>	$P \text{ AU done } Q$	Loop + invariant + decreases
<code>ensures ag(af(now(Q)))</code>	$\text{AG AF now } Q$	Ghost accumulator + decreases
<code>invariant R</code>	$\mathcal{R}$	Temporal refinement mapping
<code>decreases f</code>	f	Well-founded metric (progress)

`done(Q)`: Q holds at termination. `now(Q)`: Q holds at some intermediate state.

Example: Queue Drain (AF)

Liveness: the queue eventually becomes empty

```
fn drain(q: &mut Queue)
  requires q.view().len() > 0,
  ensures af(done(q.view().len() == 0)),
{
  while q.len() > 0
    invariant q.view().len() >= 0,
    decreases q.view().len(),
    {
      let _ = q.dequeue();
    }
}
```

$\text{invariant} = \mathcal{R}, \quad \text{decreases} = f \quad \Rightarrow \quad \text{WhileAU}_L \text{ rule applies.}$

Example: Bounded Counter (AG)

Safety: the counter stays bounded forever

```
fn bounded_counter(x: &mut u64)
  requires *x == 0,
  ensures ag(*x <= 15),
{
  loop
    invariant *x <= 10,
    {
      if *x < 10 { *x = *x + 1; }
      else      { *x = 0; }
    }
}
```

No decreases $\Rightarrow$ infinite loop $\Rightarrow$ WhileAG rule.

$\text{invariant } (*x \leq 10) \implies \text{postcondition } (*x \leq 15).$

Example: Bind Rule (AG + AF callee)

AG caller invokes AF callee — bind rule extracts postcondition

```
fn reset_to_zero(x: &mut u64)
    ensures af(done(*x == 0)),
{ *x = 0; }
```

```
fn ag_caller(x: &mut u64)
    requires *x == 0,
    ensures ag(*x <= 10),
{
    loop
        invariant *x <= 10,
        {
            if *x < 10 { *x = *x + 1; }
            if *x > 5 { reset_to_zero(x); }
        }
}
```

After reset_to_zero: assume $*x = 0$ (bind rule $\mathcal{R}$). Invariant preserved.

Example: Round-Robin Fairness (AG(AF(now)))

Every element eventually reaches the head of the queue

```
fn round_robin(queue: &mut Queue, Ghost(x): Ghost<u64>)
```

```
  requires
```

```
    queue.view().len() > 1,
```

```
    queue.view().contains(x),
```

```
  ensures
```

```
    ag(queue.view().len() > 0),
```

```
    ag(ag(now(queue.peek_spec() == x))),
```

```
{
```

```
  loop
```

```
    invariant
```

```
      queue.view().len() > 1,
```

```
      queue.view().contains(x),
```

```
    decreases queue.view().index_of_first(x),
```

```
{
```

```
  let val = queue.dequeue();
```

```
  queue.enqueue(val);
```

```
}
```

```
}
```

now() = state predicate (head = x at loop start, not end). Ghost accumulator tracks it.

Example: Lock Release Liveness (AG(AF(now)))

A held lock is always eventually released

```
fn lock_holder(lock: &mut u64, id: u64)
  requires id > 0, *lock == 0,
  ensures ag(ag(now(*lock == 0))),
{
  loop
    invariant *lock == 0 || *lock == id,
    decreases (if *lock == id { 1int } else { 0int }),
  {
    if *lock == 0 {
      *lock = id;  // acquire
      *lock = 0;   // release
    }
    if *lock == id {
      *lock = 0;   // release
    }
  }
}
```

Single-process liveness. `now(*lock == 0)` holds at release point.

Summary

Complete Rule Set

Construct	Rule
<code>*x = e</code>	State update
<code>let x = t; u</code>	BindAU _L
<code>if c {t} else {u}</code>	Case split (effectful <i>c</i>)
<code>while c {t} + decreases</code>	WhileAU _L
<code>while c {t} (no exit)</code>	WhileAG (coinductive)
<code>f($\vec{e}$)</code>	Contract-dependent

$\text{wp}(t, \varphi)$ for sequential Rust. Total correctness Hoare logic is the special case AF done Q .